

Smart Queue & Appointment Management System

CSC 2032 – Object Oriented Programming



Group Members :

AS20240479

M.P.G.D.Nimeshika

AS20240596

O.A.Thewarapperuma

GitHub repository link : <https://github.com/Dilki30/smart-queue-system.git>

Content

1. Introduction	1
2. Problem Statement	1
3. Objectives.....	1
3.1 Main Objective	1
3.2 Specific Objectives	1
4. Proposed Solution	2
5. Technologies Used.....	2
6. System Features	2
6.1 User Authentication	2
6.2 Dashboard	3
6.3 Appointment Booking	3
6.4 Queue Management	3
6.5 Ticket Generation	3
6.6 History	3
6.7 User Profile.....	4
6.8 Admin Authentication	4
7. Object-Oriented Programming Concepts.....	4
7.1 Encapsulation	4
7.2 Abstraction	4
7.3 Inheritance	5
7.4 Polymorphism	5
8. System Architecture	5
9.1 User Table.....	6
9.2 Appointment Table.....	6
9.3 Queue Table	6
10. System Interfaces	7
11. Testing	12
12. Future Improvements	12
13. Conclusion.....	13
14. References.....	13

1. Introduction

Queue management has become a significant challenge in hospitals, banks, government offices, and other public service organizations. Traditional manual queue systems cause long waiting times, overcrowding, and poor customer satisfaction.

The **Smart Queue Appointment System** is a web-based application developed to digitalize queue management by allowing customers to reserve appointments online, receive queue numbers, monitor queue status, and view their appointment history.

2. Problem Statement

Many organizations still rely on manual token systems.

Problems include:

- Long waiting times
- Overcrowded waiting areas
- Customers unsure about their queue position
- Time wastage
- Difficult queue management
- No online booking system
- No appointment history

3. Objectives

3.1 Main Objective

Develop an online Smart Queue Appointment System using Object-Oriented Programming concepts.

3.2 Specific Objectives

- Allow users to register and login
- Book appointments online
- Generate queue tokens automatically
- Display live queue information
- Maintain appointment history
- Improve customer experience
- Reduce waiting time

4. Proposed Solution

The proposed system provides an online platform where customers can:

- Create an account
- Login securely
- Select a service
- Book appointments
- Receive queue numbers
- View live queue status
- Access booking history
- Manage their profile

Administrators can manage queues and appointments efficiently.

5. Technologies Used

Component	Technology
Frontend	HTML5
Styling	CSS3
Client-side	JavaScript
Backend	Node.js
Architecture	REST API
Data Storage	JSON File Database (Can be upgraded to MySQL)
Version Control	Git & GitHub

6. System Features

6.1 User Authentication

- User Registration
- User Login
- Logout
- Session Management

6.2 Dashboard

Displays

- User information
- Upcoming appointments
- Queue statistics
- Navigation panel

6.3 Appointment Booking

Users can

- Choose service
- Select appointment date
- Select appointment time
- Submit booking

6.4 Queue Management

System automatically

- Generates queue numbers
- Displays queue position
- Shows waiting status
- Updates queue progress

6.5 Ticket Generation

After booking

- Queue number generated
- Appointment details shown
- Booking confirmation displayed

6.6 History

Users can

- View previous bookings
- View completed appointments
- Track appointment records

6.7 User Profile

Users can update

- Name
- Email
- Password
- Personal information

6.8 Admin Authentication

Admin can view queue status and cancel appointments.

7. Object-Oriented Programming Concepts

The project follows Object-Oriented Programming principles.

7.1 Encapsulation

Sensitive user information is managed securely through classes and modules.

Example:

- User class
- Ticket Handler
- Authentication Handler

7.2 Abstraction

Complex backend operations are hidden from users.

Examples:

- Queue generation
- Authentication
- Booking process

7.3 Inheritance

Future versions can include:

User
↓
Customer
↓
Administrator

Both inherit common properties from the User class.

7.4 Polymorphism

Different service handlers perform booking differently.

Examples

- Hospital Handler
- Bank Handler
- Ticket Handler

Each processes requests uniquely while using a common structure.

8. System Architecture



9. Database Design

9.1 User Table

Field	Type
UserID	Integer
Name	Varchar
Email	Varchar
Password	Varchar

9.2 Appointment Table

Field	Type
AppointmentID	Integer
UserID	Integer
Service	Varchar
Date	Date
Time	Time
QueueNumber	Integer
Status	Varchar

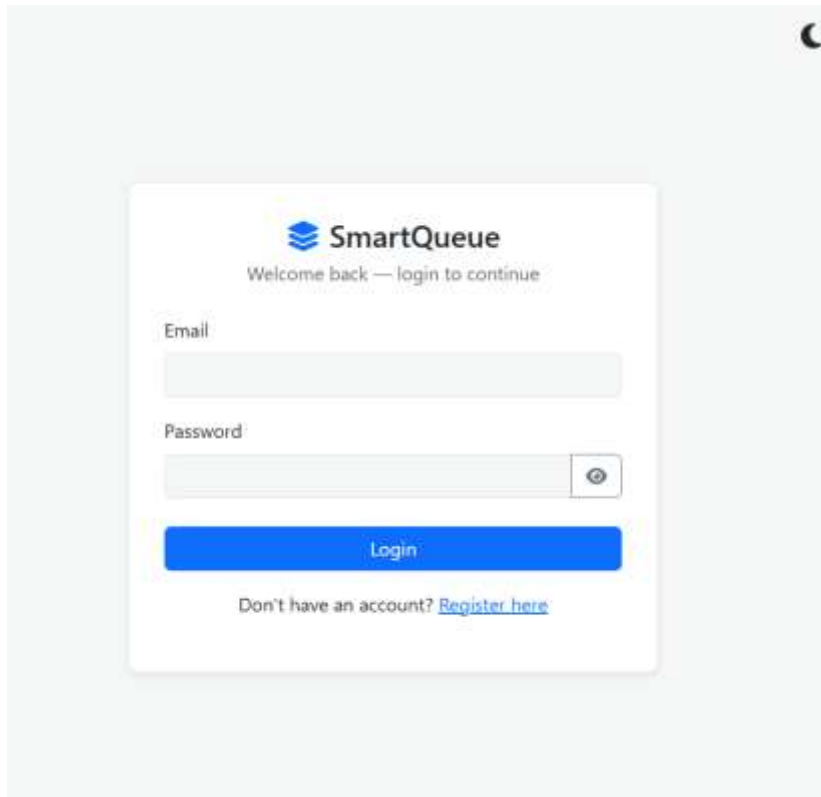
9.3 Queue Table

Field	Type
QueueID	Integer
QueueNumber	Integer
CurrentPosition	Integer
Status	Varchar

10. System Interfaces

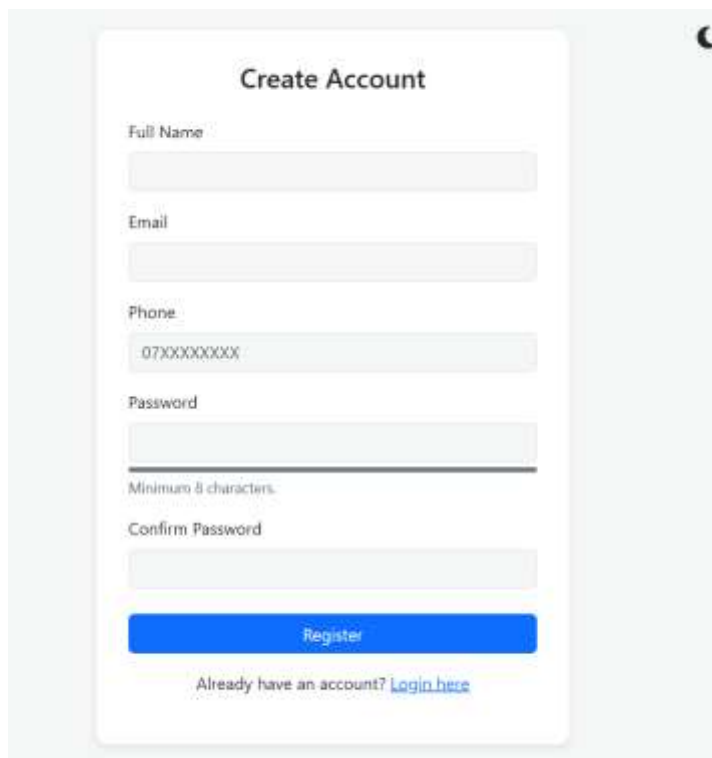
The project contains the following pages:

- Login Page



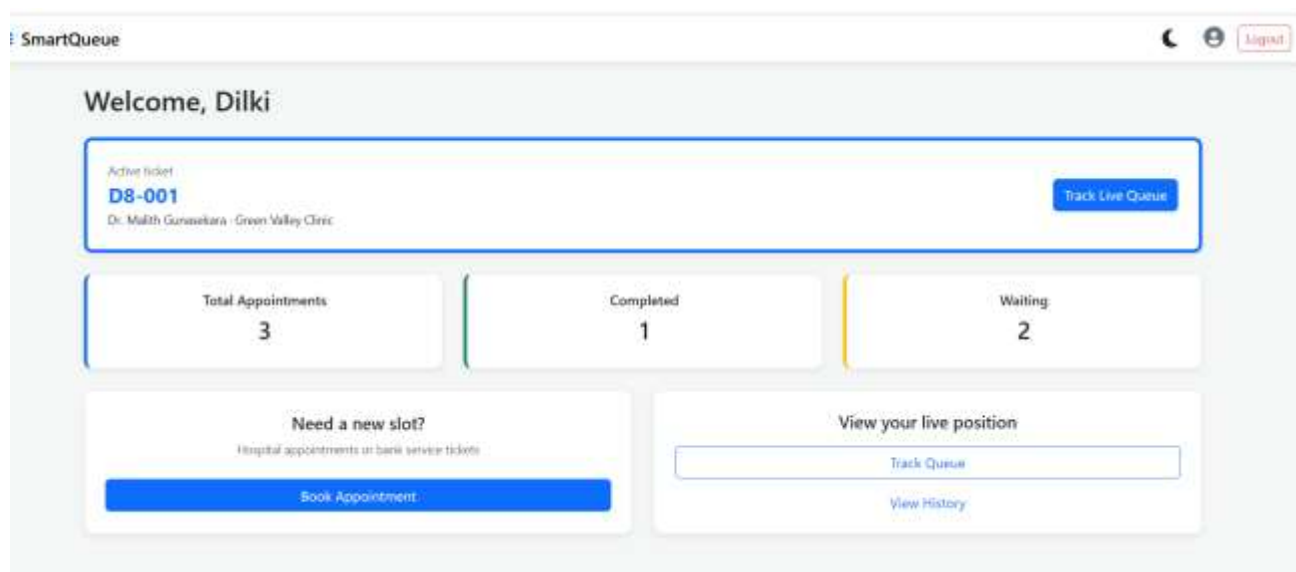
The image shows a login page for SmartQueue. It features a white card on a light gray background. At the top of the card is the SmartQueue logo and the text "Welcome back — login to continue". Below this are two input fields: "Email" and "Password". The "Password" field has a toggle icon on the right. A blue "Login" button is positioned below the input fields. At the bottom of the card, there is a link: "Don't have an account? [Register here](#)". A dark gray moon icon is visible in the top right corner of the background.

- Registration Page

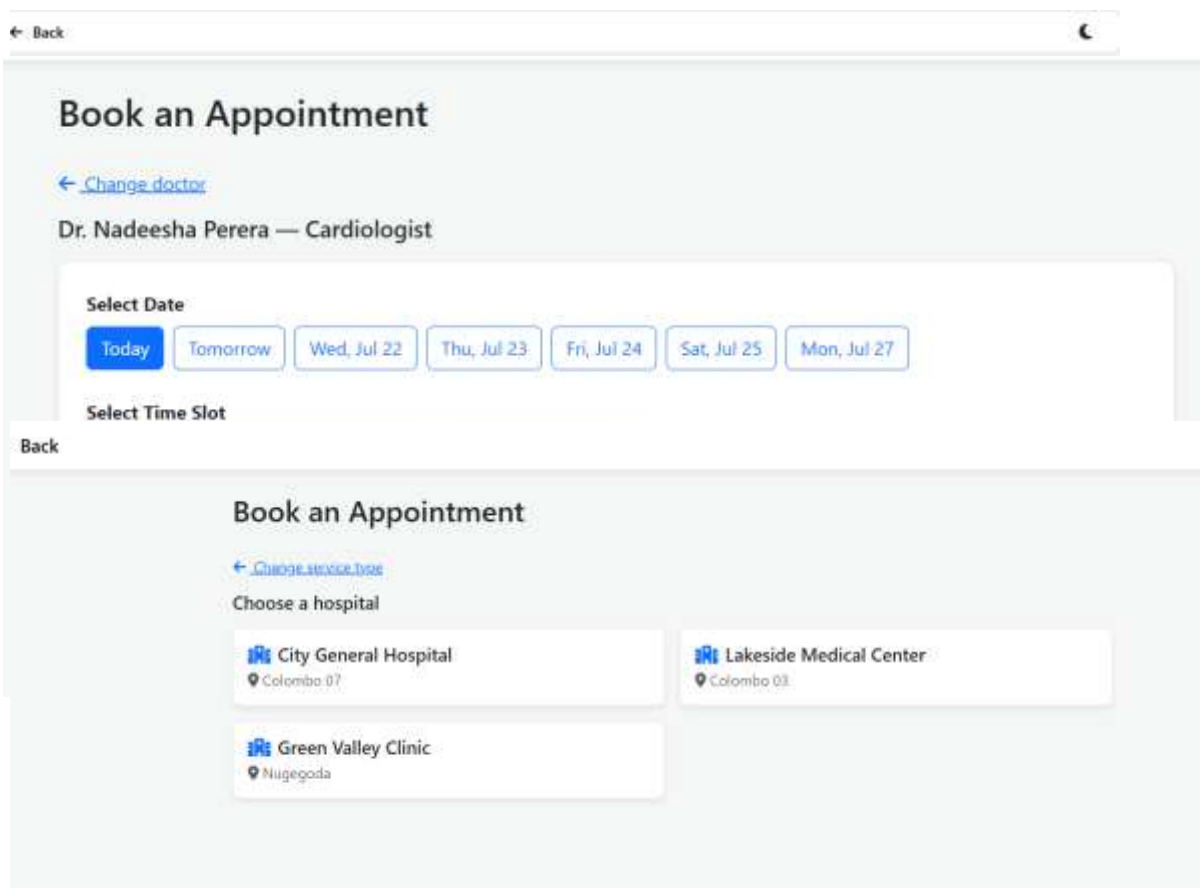


The image shows a registration page for SmartQueue. It features a white card on a light gray background. At the top of the card is the title "Create Account". Below this are five input fields: "Full Name", "Email", "Phone" (with a placeholder "07XXXXXXXX"), "Password", and "Confirm Password". Below the "Password" field, there is a note: "Minimum 8 characters". A blue "Register" button is positioned below the input fields. At the bottom of the card, there is a link: "Already have an account? [Login here](#)". A dark gray moon icon is visible in the top right corner of the background.

- Dashboard



- Appointment Booking



Book an Appointment

[← Change hospital](#)

City General Hospital

Type a disease, symptom, or specialization to find the right doctor.

🔍 e.g. heart, skin, tooth, fever...

 **Dr. Nadeesha Perera**

Cardiologist

[heart](#) [chest pain](#) [high blood pressure](#) [hypertension](#)

 **Dr. Kasun Silva**

Dermatologist

[skin](#) [acne](#) [rash](#) [eczema](#)

 **Dr. Amaya Fernando**

Pediatrician

[child](#) [children](#) [fever](#) [flu](#)

- Queue Status

Your Token

D8-001

Position

1

Est. Wait

25m

 Refresh

 Almost your turn — please be ready!

- Queue Ticket

SmartQueue Ticket

City General Hospital

D1-001

Doctor:

Dr. Nadeesha Perera

Specialty:

Cardiologist

Date:

2026-07-20

Time:

09:30

Position:

1

Est. wait:

15 min

Issued:

7/19/2026, 7:54:38 PM

Please arrive 10 minutes before your position is called.

- Booking History

Appointment History

Token	Type	Details	Date	Status	
R1-001	 Bank	Cash Deposit	2026-07-19	Completed	View
D8-001	 Hospital	Dr. Malith Gunasekara	2026-07-23 10:30	Almost	View
D1-001	 Hospital	Dr. Nadeesha Perera	2026-07-20 09:30	Completed	View

- User Profile

My Profile

Name	REDACTED	Phone	0771234567
Email	REDACTED		

Update Password

Current Password			
New Password		Confirm Password	

[Save Changes](#)

- Admin Login



Welcome back — login to continue

Email

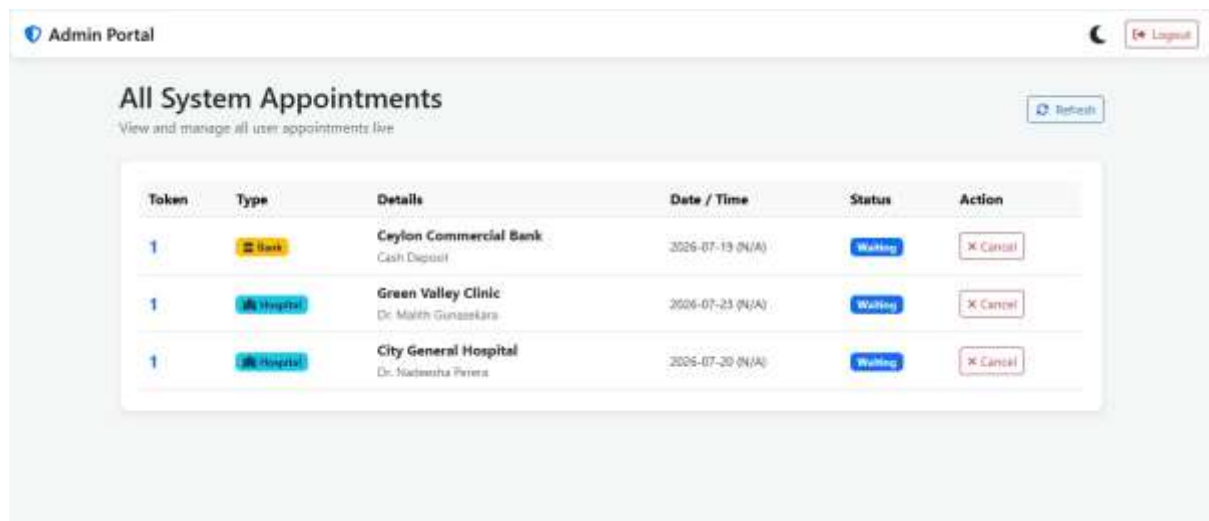
Password



[Login](#)

Don't have an account? [Register here](#)

- Admin Portal



11. Testing

Test Case	Expected Result	Status
User Registration	New account created	Passed
User Login	Login successful	Passed
Book Appointment	Appointment created	Passed
Queue Generation	Queue number assigned	Passed
View Queue	Queue displayed	Passed
View History	Booking history shown	Passed
Logout	Session terminated	Passed

12. Future Improvements

Future enhancements include:

- MySQL database integration
- Java Spring Boot backend
- Email notifications
- SMS reminders
- QR code tickets

- AI-based waiting time prediction
- Mobile application (Android & iOS)
- Online payment integration
- Admin analytics dashboard
- Real-time queue updates using WebSockets

13. Conclusion

The Smart Queue Appointment System successfully addresses the inefficiencies of traditional queue management by providing a web-based digital solution. Users can conveniently register, book appointments, receive queue numbers, monitor queue status, and access booking history. The project demonstrates the effective use of Object-Oriented Programming principles, modern web technologies, and modular system design, making it scalable for future enhancements and suitable for real-world deployment.

14. References

1. Oracle Java Documentation
2. MDN Web Docs (HTML, CSS, JavaScript)
3. Node.js Documentation
4. GitHub Documentation
5. REST API Design Guidelines

